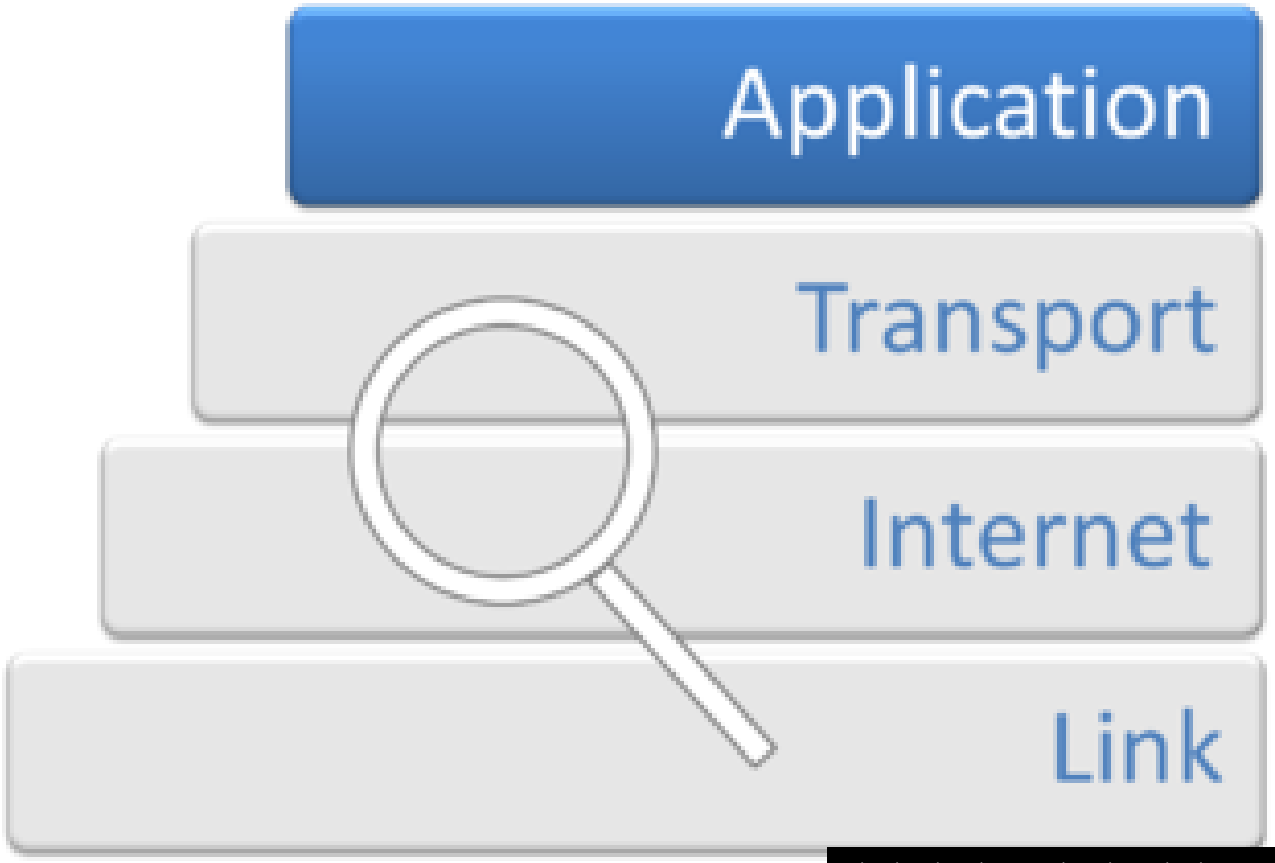
Application Layer: HTTP

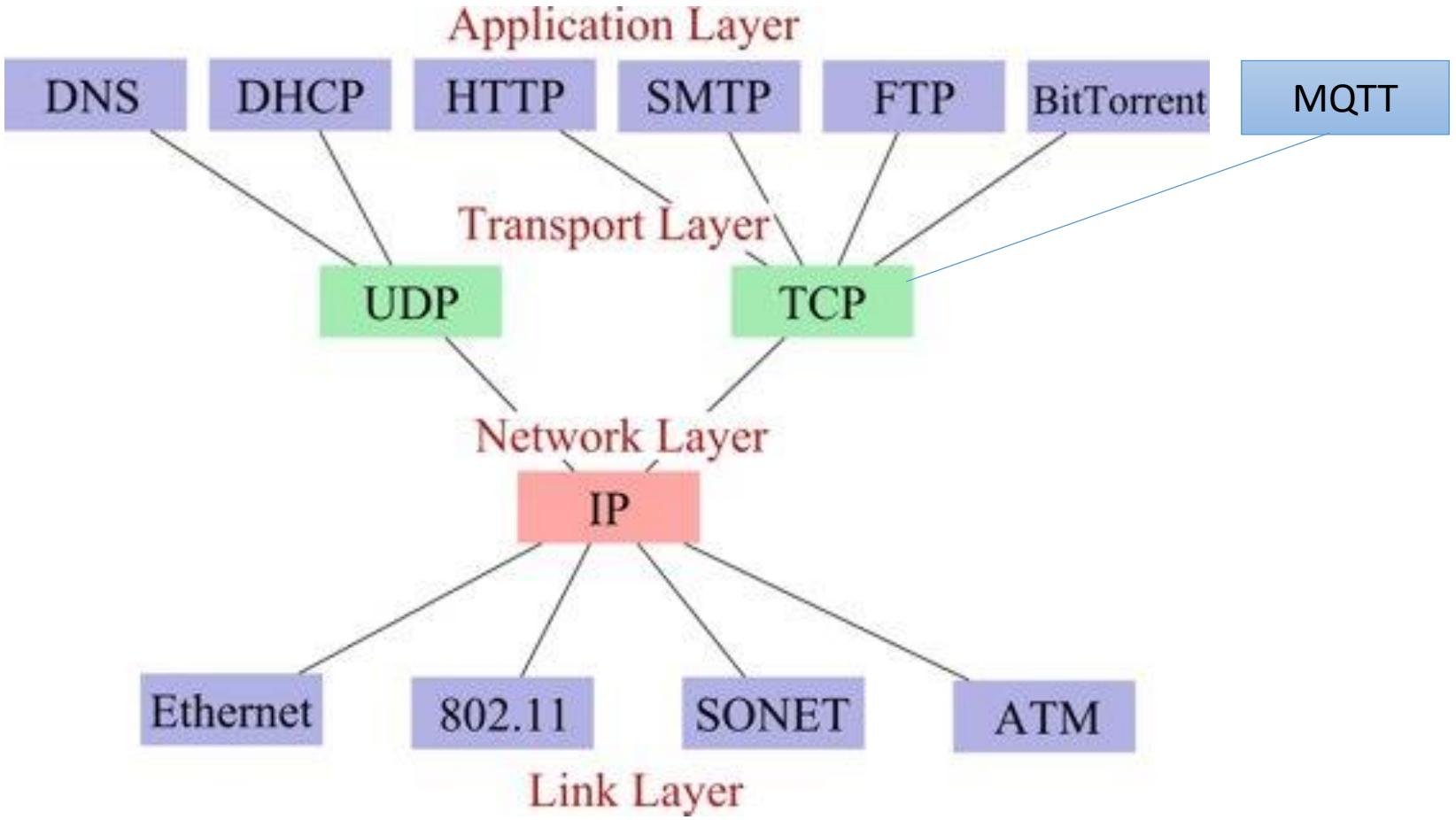
Frank Walsh

Application Layer

- The application layer provides the interfaces and protocols needed by users/applications to access the network
  - It facilitates the users/network-based applications to use the services of the network.
  - Provides services/protocols that support user authentication, naming network devices, formatting messages, and e-mails, transfer of files etc.

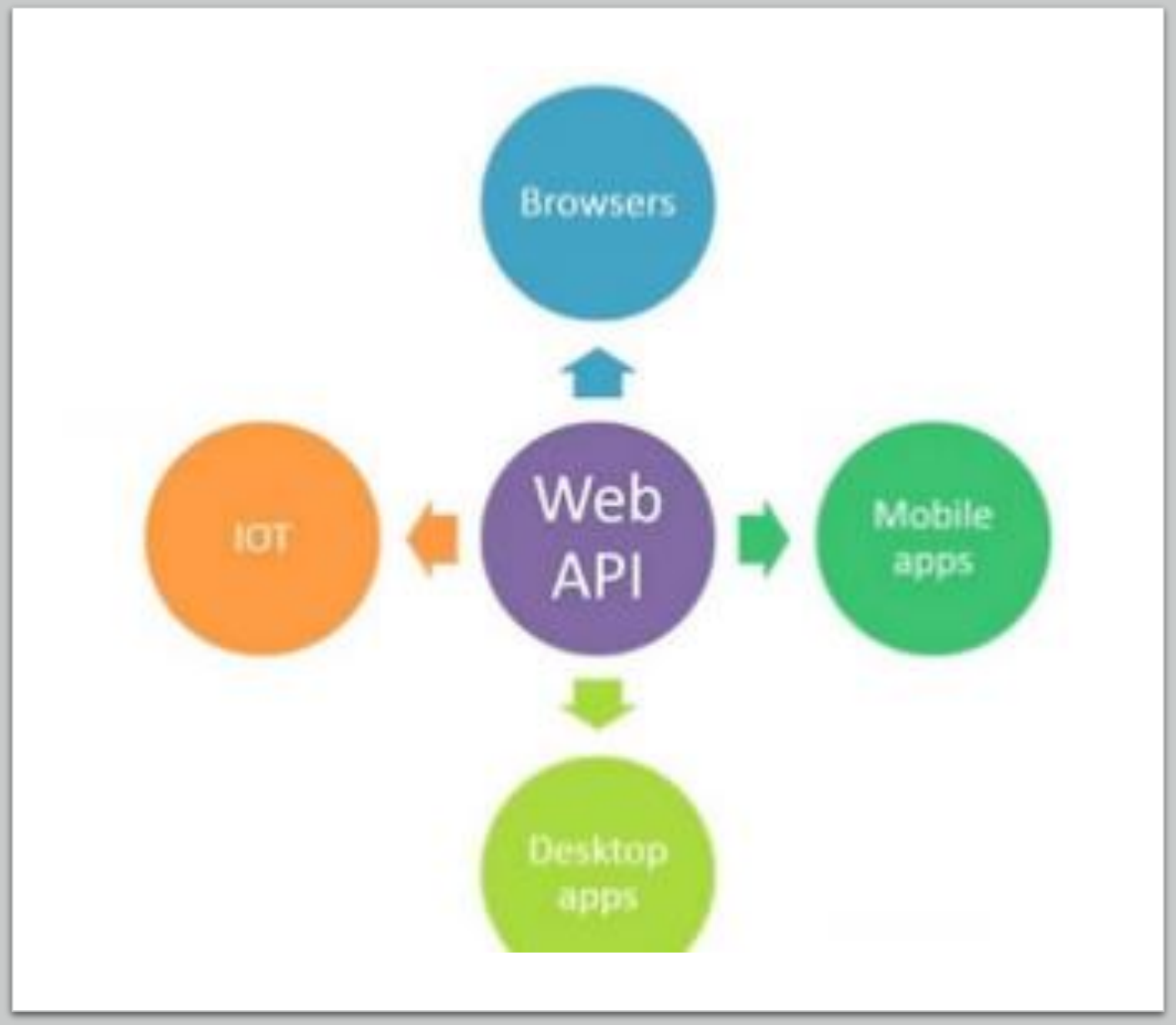


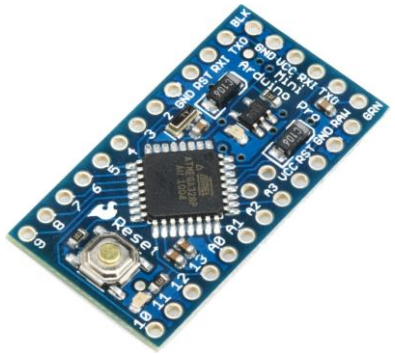
TCP/IP Protocol Stack

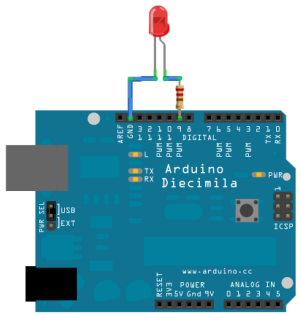


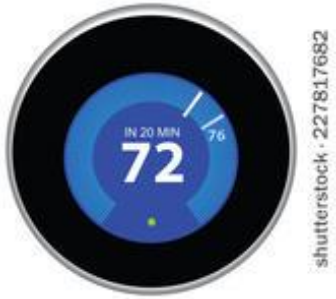
Web APIs

- Programmatic interface exposed via the web
- Uses open standards typically with request-response messaging.
  - Messages in well known format(e.g. JSON)
  - HTTP as transport
  - URLs
- Example would be Restful web service
- Typical use:
  - Expose application functionality via the web
  - Machine to machine communication
  - Distributed systems







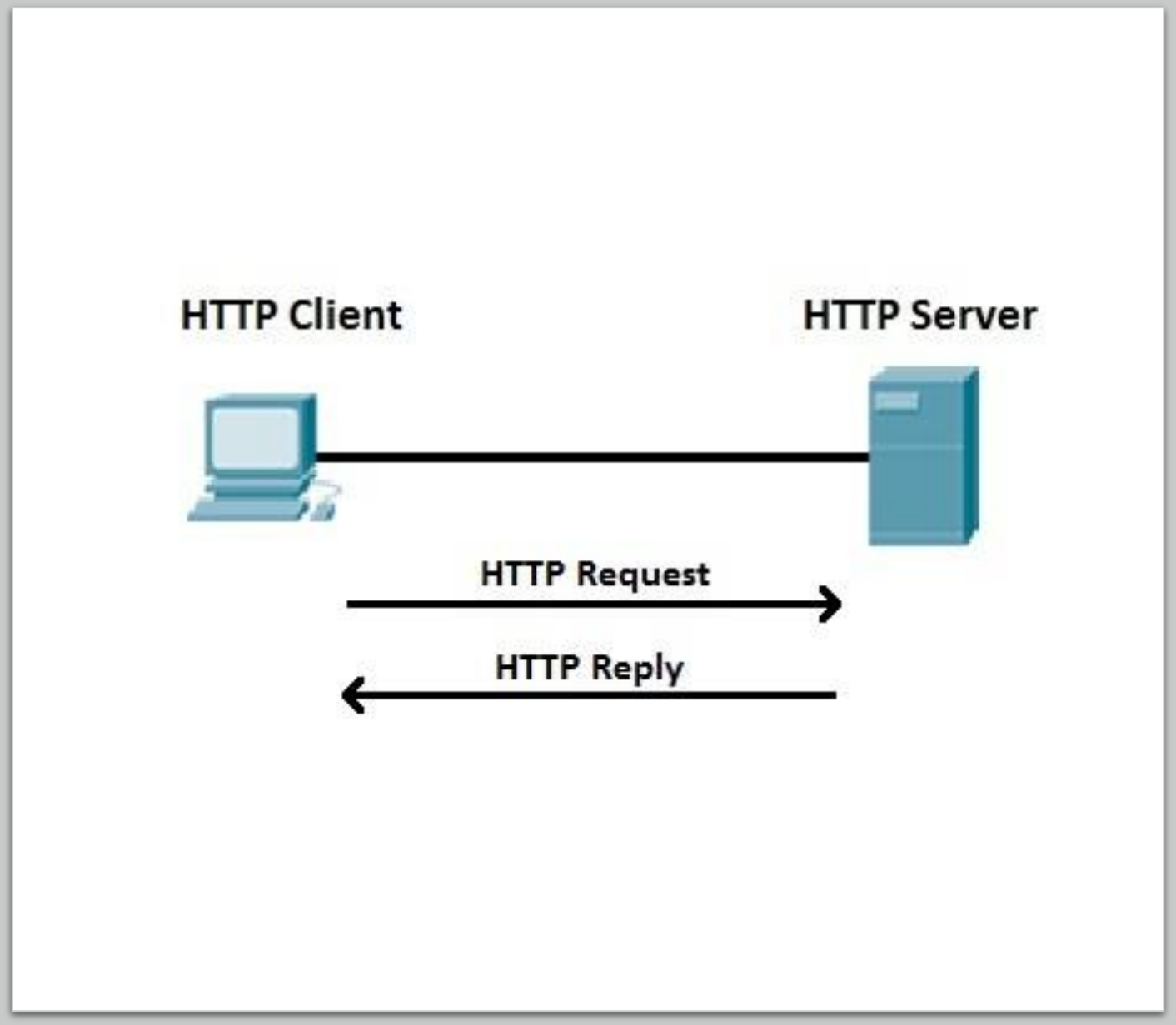


What's a Web API

- Typically implements HTTP
  - processes HTTP requests
- Usually runs on machine connected to a network
  - Has an IP address
- Serves up resources
  - Ideally in a "restful" manner
- Many different types of devices can run a web API...

What’s HTTP

- HyperText Transfer Protocol
- Protocol used in World Wide Web
  - **http**://www.setu.ie
- Your browser communicates using HTTP (HTTP Client)
- Devices communicate using HTTP
- Simple, ubiquitous.

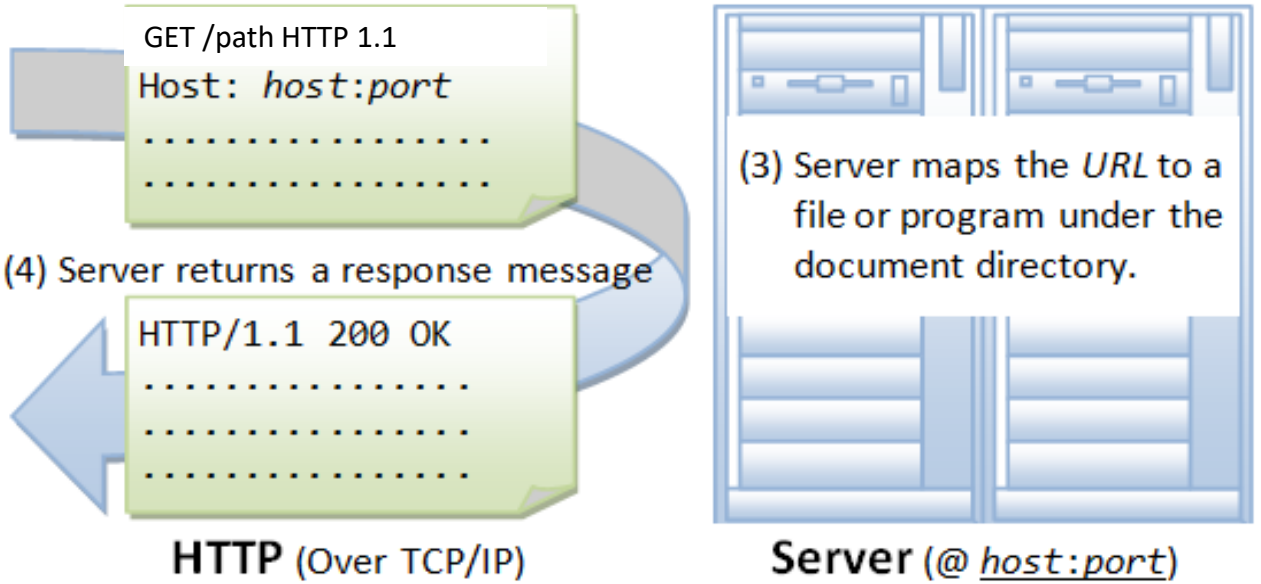


HTTP request from an application.

(1) **URL** used to create HTTP request
http://host:port/path/to/resource

(5) App processes response

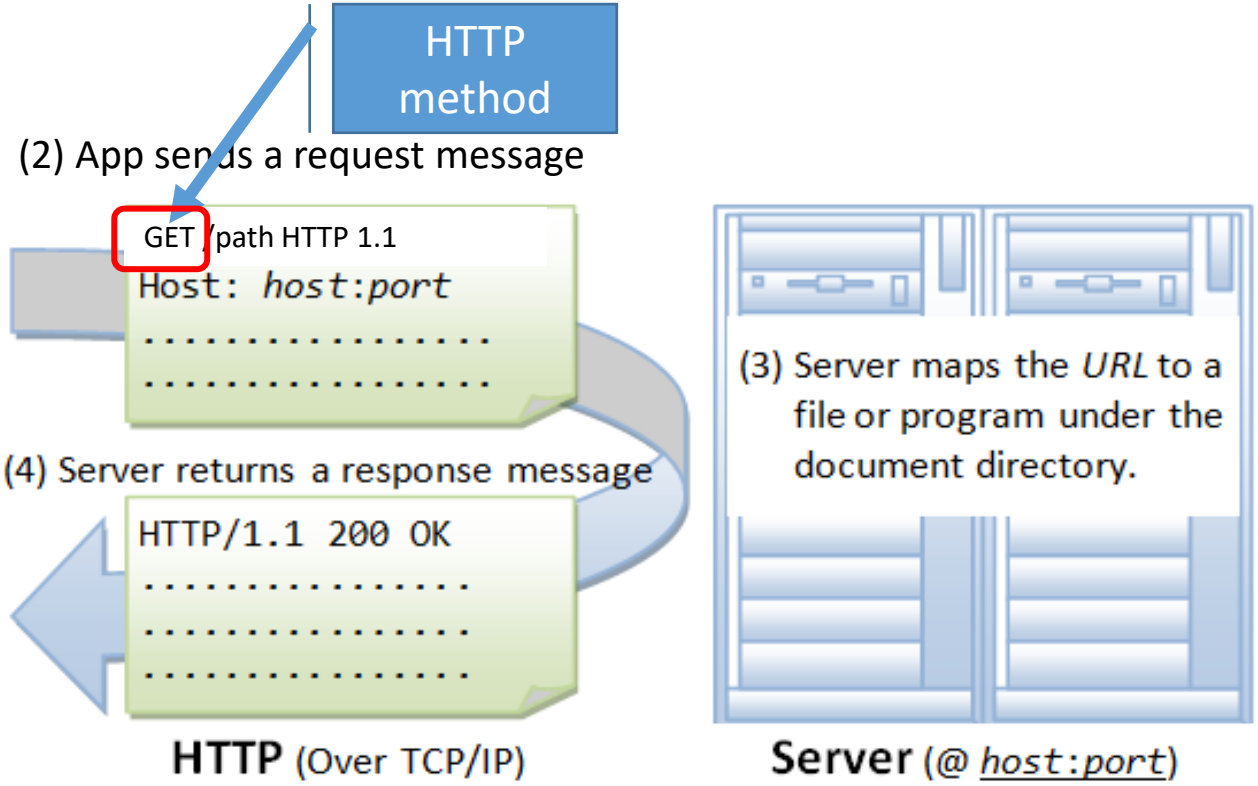
(2) App sends a request message



HTTP: The Method/Verb

(1) URL used to create HTTP request
http://host:port/path/to/resource

(5) App processes response



HTTP: **The URL**

- A URL (Uniform Resource Locator) uniquely identifies a resource over the web. *protocol://hostname:port/path*
- There are 4 parts in a URL:
  - ***Protocol***: The application-level protocol used by the client and server, e.g., HTTP, FTP, and telnet.
  - ***Hostname***: The domain name (e.g., www.nowhere123.com) or IP address (e.g., 192.128.1.2) of the server.
  - ***Port***: The TCP port number that the server is listening for incoming requests from the clients.
  - ***Path-and-file-name***: The name and location of the requested resource, under the server document base directory.
- Example, for http://www.nowhere123.com/docs/index.html
  - the communication protocol is HTTP
  - he host is www.nowhere123.com.
  - The port number was not specified, and takes default number, which is TCP port 80 for HTTP.
  - The path for the resource to be located is "/docs/index.html".

HTTP: The Request

- HTTP clients (e.g. a browser or Python module) translates a URL into a request message according to the specified protocol; and sends the request message to the server.
- For example, the client translated the URL http://www.nowhere123.com/doc/index.html into the following request message:

```
GET /docs/index.html HTTP/1.1
Host: www.nowhere123.com
Accept: image/gif, image/jpeg, */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)
(blank line)
```

HTTP: The Response

- When this request message reaches the server, the server can take either one of these actions:
  1. The server interprets the request received, maps the request into a file under the server's document directory, and returns the file requested to the client.
  2. The server interprets the request received, maps the request into a program kept in the server, executes the program, and returns the output of the program to the client.
  3. The request cannot be satisfied, the server returns an error message.

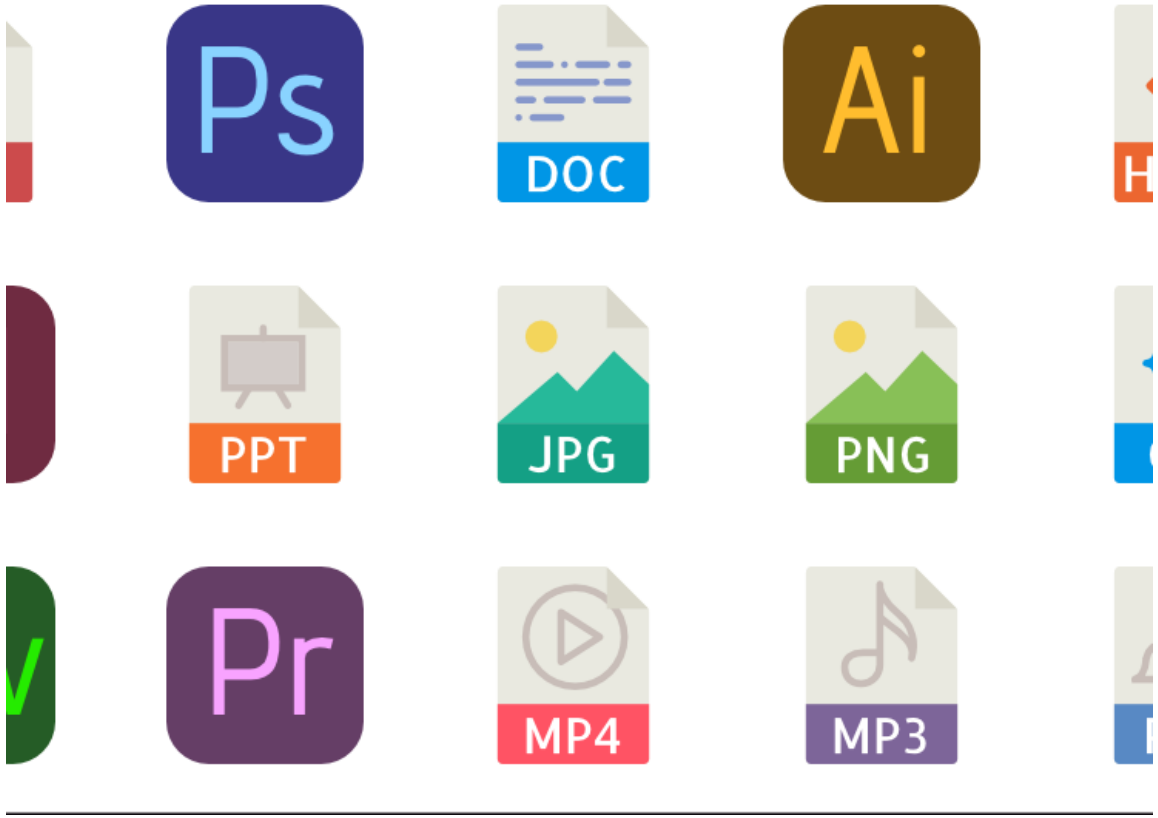
An example of the HTTP response message is below:

HTTP/1.1 200 OK
Date: Sun, 18 Oct 2009 08:56:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Sat, 20 Nov 2004 07:16:26 GMT
Content-Length: 44
Connection: close
Content-Type: text/html

<html><body><h1>It works!</h1></body></html>

HTTP Protocol: **Content-Type Header**

- The Content-Type tells the client what the content type of the returned content.
- Also known as “MIME” ,”media type”, "content type")
  - a video file might be **audio/mpeg**, or an image file **image/png**).
- The Internet Assigned Numbers Authority (IANA) is the official authority for the standardization and publication of these classifications.



HTTP: **The URL Query String**

- Query string used to include data in a URL. For example

- The server can use the query string to execute logic associated with that resource. In this example, it could be used to set the status of the resource (heating) to true.
- Remember OpenWeather API…

HTTP Methods ("typical uses")

GET

- Request resource without sending data

PUT

- Modify/replace resource with data that you are sending

POST

- Create new resource with data that you are sending

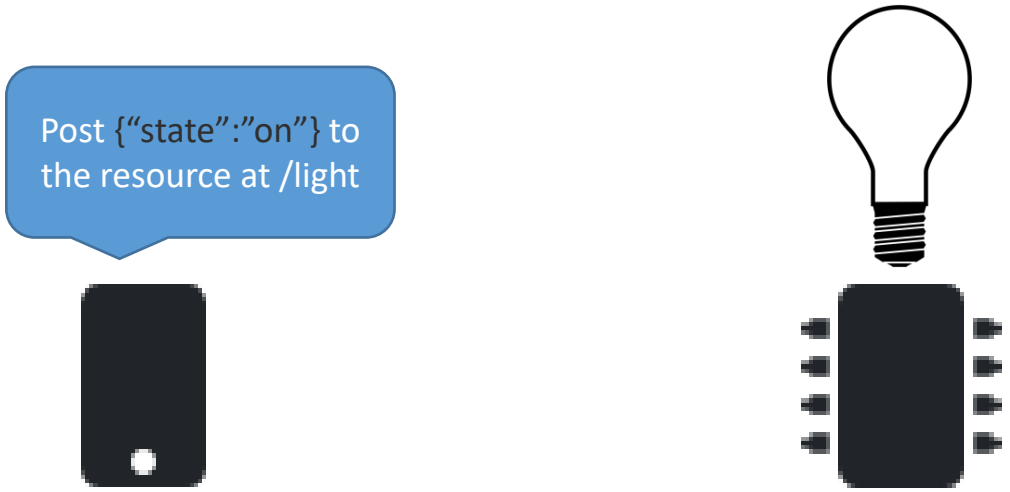
DELETE

- Delete resource without sending data

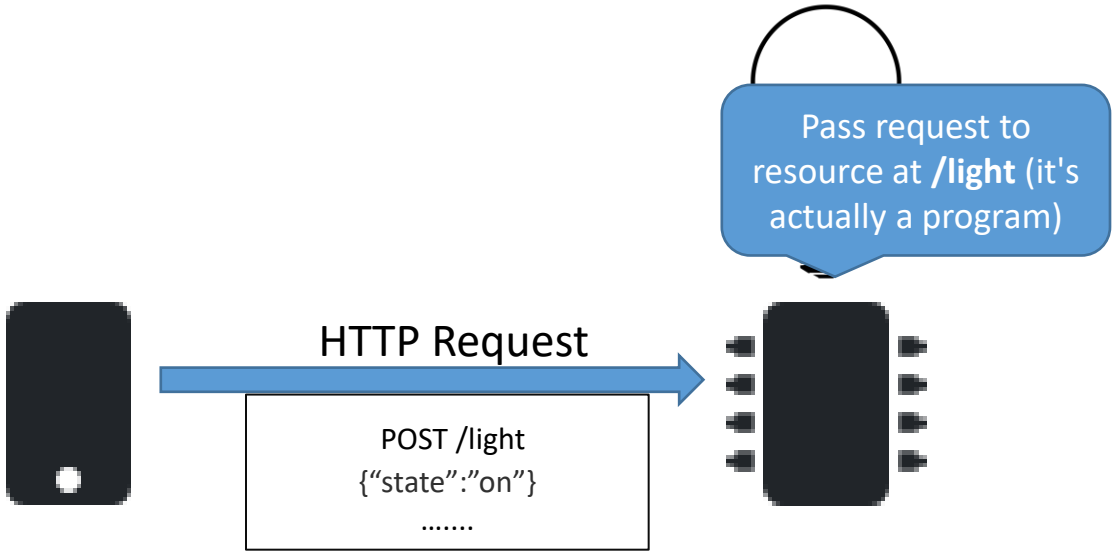
HTTP: The Body

- An optional *body* containing data associated with the message.
- **For a HTTP request, the body can contain:**
  - content of an HTML form
  - **commands for a device**
- **For a HTTP response, the body can be:**
  - data associated with a response.
- The presence of the body and its format is specified by the start-line and HTTP headers.

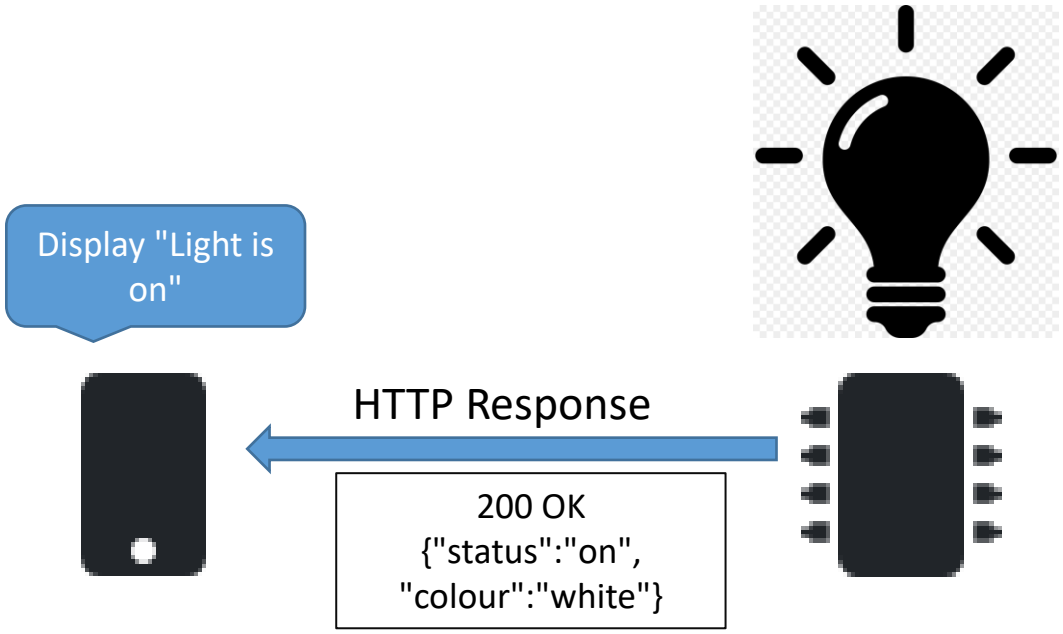
HTTP API for IOT Example



HTTP Post


Pass request to resource at **/light** (it's actually a program)
HTTP Request
POST /light
{“state”:”on”}
…….

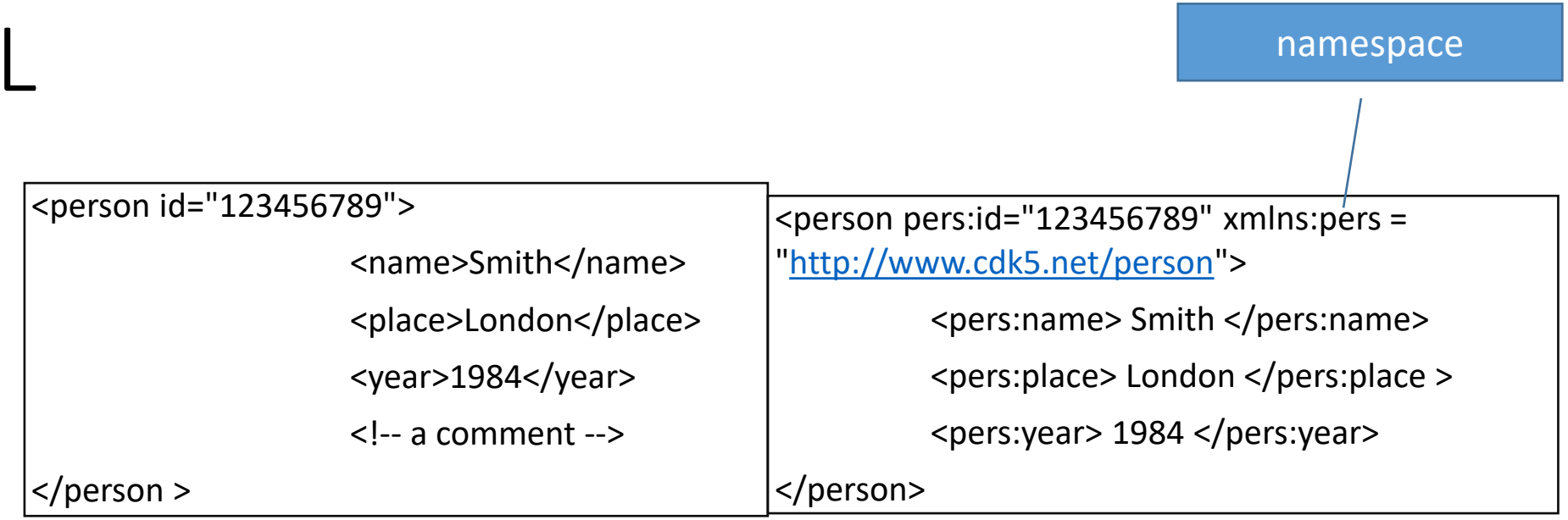
HTTP Post


Display "Light is on"
HTTP Response
200 OK
{"status":"on",
"colour":"white"}

External Data Representations
XML

- eXtensible Markup Language(XML)
- Same heritage as HTML(but XML is NOT HTML)
- XML data items are tagged with ‘markup’ strings
  - used to describe the logical structure of the data
- XML has many uses. For now we confine ourselves to external data representations
- Has many cool features including
  - Extensible
  - Textual
  - Kind of human readable and machine readable...

XML



- Above shows XML definitions of the Person structure.
  - As with xHTML, tags enclose character data.
  - Tags : <name>, <place>,<year> data:”Smith”, “London”…
- Namespaces provide a means for scoping names

JSON

- JavaScript Object Notation
- Lightweight text-based open standard designed for human readable data interchange.
- Can represent simple data structures and associative arrays.
- Good for serializing and transmitting structured data across a network


```
{
    "id":"example",
    "current_value":"500",
    "at":"2013-05-06T00:30:45.694188Z",
    "max_value":"500.0",
    "min_value":"333.0",
    "version":"1.0.0"
}
```

JSON

- JSON is a data interchange format technique
- A collection of name/value pairs.
- Application programming interfaces(APIs) exist for most programming languages

```
{
  "person": {
    "id": "123456789",
    "name": "Smith",
    "place": "London",
    "year": "1984"
  }
}
```

```
{ "temperature" : 72.55 }
```

HTTPS

- HTTP is not a secure protocol
  - HTTPS (Secure HTTP) should be used for internet communications
- Components:
  - HTTP
  - SSL/TLS: Secure Sockets Layer (SSL) or Transport Layer Security (TLS) encrypts the data between the client and server
- Features:
  - **Data Encryption**: Protects sensitive information from being intercepted
  - **Authentication**: Verifies the identity of websites, preventing “man-in-the-middle” attacks

More on HTTP

- For an extensive overview, checkout:

https://www.ntu.edu.sg/home/ehchua/programming/webprogramming/http_basics.html